✅ CORPORATE VPD PROJECT – FULL SETUP SCRIPT (Till Step 8)

🔹 1️⃣ Create Database

```
CREATE DATABASE corporate_db;

\c corporate_db;
```

🔹 2️⃣ Create Tables

Roles Table

```
CREATE TABLE roles (
    role_name VARCHAR(20) PRIMARY KEY,
    description TEXT
);
```

Users Table

```
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role VARCHAR(20) REFERENCES roles(role_name),
    department VARCHAR(50),
    location VARCHAR(20)
);
```

Employees Table (VPD Target)

```
CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    department VARCHAR(50),
    salary NUMERIC,
    created_by VARCHAR(50),
    sensitivity_level INT
);
```

Sessions Table

```
CREATE TABLE sessions (
    session_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    login_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ip_address VARCHAR(50),
    active BOOLEAN DEFAULT TRUE
);
```

Audit Logs Table

```
CREATE TABLE audit_logs (
    audit_id SERIAL PRIMARY KEY,
    user_id INT,
    action VARCHAR(20),
    table_name VARCHAR(50),
    access_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    result VARCHAR(20)
);
```

🔹 3️⃣ Enable Row-Level Security (VPD Equivalent)

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;

ALTER TABLE employees FORCE ROW LEVEL SECURITY;
```

🔹 4️⃣ Insert Corporate Roles

```
INSERT INTO roles (role_name, description) VALUES
('Admin', 'System administrator with full access'),
('Manager', 'Department-level manager'),
('HR', 'Human resources staff'),
('Employee', 'Regular employee'),
('Auditor', 'Read-only auditor');
```

🔹 5️⃣ Insert Sample Users

```
INSERT INTO users (username, password_hash, role, department, location) VALUES
('admin1', 'hash_admin', 'Admin', 'IT', 'Internal'),
('manager1', 'hash_manager', 'Manager', 'Sales', 'Internal'),
('hr1', 'hash_hr', 'HR', 'HR', 'Internal'),
('emp1', 'hash_emp', 'Employee', 'Sales', 'Internal'),
('auditor1', 'hash_audit', 'Auditor', 'Audit', 'Internal');
```

🔹 6️⃣ Insert Sample Employee Data

```
INSERT INTO employees (emp_name, department, salary, created_by, sensitivity_level) VALUES
('Alice', 'Sales', 70000, 'hr1', 2),
('Bob', 'Sales', 65000, 'hr1', 2),
('Charlie', 'IT', 90000, 'admin1', 3),
('David', 'HR', 60000, 'hr1', 3),
('Eve', 'Sales', 50000, 'hr1', 1);
```

🔹 7️⃣ Create Application Database Roles (Non-Superuser)

```
CREATE ROLE app_admin LOGIN;
CREATE ROLE app_manager LOGIN;
CREATE ROLE app_hr LOGIN;
CREATE ROLE app_employee LOGIN;
CREATE ROLE app_auditor LOGIN;
```

🔹 8️⃣ Grant Minimum Required Privileges

```
GRANT CONNECT ON DATABASE corporate_db TO
app_admin, app_manager, app_hr, app_employee, app_auditor;

GRANT USAGE ON SCHEMA public TO
app_admin, app_manager, app_hr, app_employee, app_auditor;

GRANT SELECT, INSERT, UPDATE ON employees TO
app_admin, app_manager, app_hr;

GRANT SELECT ON employees TO app_employee;

GRANT SELECT ON audit_logs TO app_auditor;
```

🔹 9️⃣ Session Context Variables (Example: HR Login)

```
SET app.username = 'hr1';
SET app.role = 'HR';
SET app.department = 'HR';
SET app.location = 'Internal';
```

You can verify:

```
SHOW app.role;
SHOW app.department;
SHOW app.location;
```

🔹 🔟 Create Row-Level Security Policies

HR Policy

```
CREATE POLICY hr_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'HR'
    AND department = current_setting('app.department', true)
);
```

Manager Policy

```
CREATE POLICY manager_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Manager'
    AND department = current_setting('app.department', true)
);
```

Employee Policy

```
CREATE POLICY employee_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Employee'
    AND emp_name = current_setting('app.username', true)
);
```

Admin Policy

```
CREATE POLICY admin_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Admin'
);
```

✅ What You Have Now

You have successfully implemented:

- Database schema
- Role-Based Access Control
- Context binding via session variables
- Virtual Private Database (Row-Level Security)
- Role-specific data filtering

You now have:

> Same query → Different results → Based on role & context

This is the **core deliverable of your project**.

When you're ready, next we will:

🔐 Properly test using non-superuser roles
📊 Add Time-based + Location-based context
📝 Implement full auditing with triggers

Say **next**.